THE LEGENDARY REFRIGERATOR

냉장고, 전설이 되다

냉장고 사진 한 장으로 낭비 없는 식탁을 만드는 AI

딥러닝 프로젝트 발표



01 INTRODUCTION

텅 빈 냉장고 앞의 매일 반복되는 고민

“ 오늘 뭐 먹지 ?”

사소하지만 매일 반복되는 고민을 딥러닝으로

식재료 인식

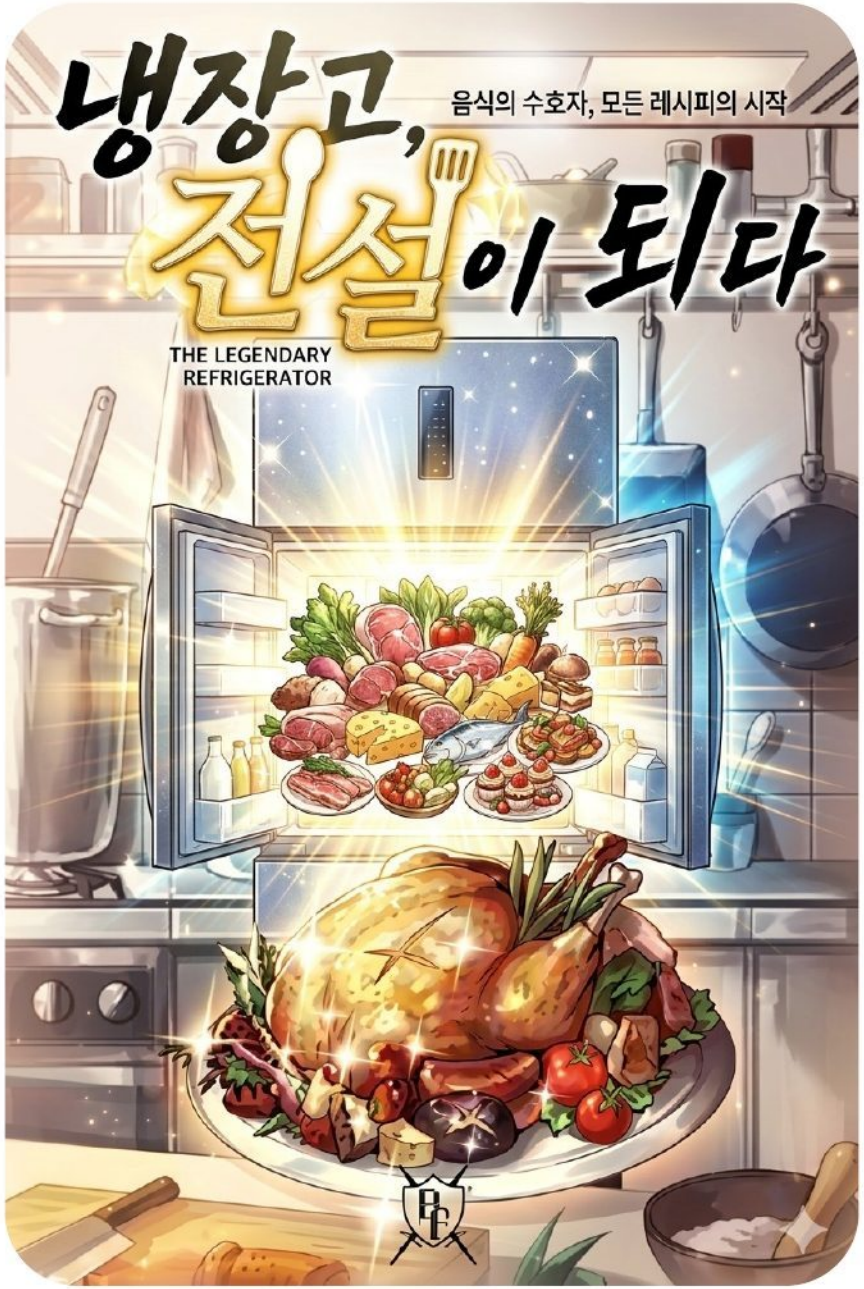
냉장고 속 재료를 사진 한 장으로 자동 탐지

맞춤 추천

건강 상태와 냉장고 상황에 맞는 최적 레시피 제안

낭비 절감

음식 낭비를 줄이고 바른 식생활을 돕는 실용 시스템



3 단계 데이터 파이프라인 구축

페르소나 “미국으로 교환학생을 간 배고픈 주희” (건강 설계) — 서양식 식재료 위주로 구성되어 있어 , 한국적인 식단 환경과는 차이



3 단계 데이터 파이프라인 구축

STEP 1

데이터 신뢰성 확보

파편화된 3 개 데이터셋을
인덱스 재매핑 · 충돌방지로 통합
23 만 건 레시피 중 노이즈
1.4 만여 개 제거 (제로 전처리)

```
================================================
🧹 레시피 전처리 & 이상값 필터링
================================================
필터 전: 231,637개
  재료 ≥ 3개 필터 후:      229,485개
  조리시간 0분 제거 후:    228,397개 (제거: 1,088개)
  조리시간 ≤ 300분 후:     218,570개 (제거: 9,827개)
  칼로리 0~5000 후:        217,603개 (제거: 967개)
  재료 ≤ 30개 필터 후:     217,575개 (제거: 28개)
  이름·중복 정리 후:       217,574개 (제거: 1개)

✅ 최종 사용 가능 레시피: 217,574개 (93.9% 유지)
```

STEP 2

정규화 · 데이터 증강

data.yaml 로 train/val/test 클래스 표준화
정규표현식 + 매핑 사전으로 명칭 통일
조명 · 그림자 · 흔들림 등 실제
주방 환경 반영한 증강 파이프라인

```python
# 7-5. 세 데이터셋 필터링
print(f"\n{'='*50}")
print(f"📦 SRC_ROOT 필터링...")
print(f"{'='*50}")
actual_splits = {}
for s in ["train", "valid", "test"]:
    actual_splits[s] = filter_split(SRC_ROOT, old_to_new1, s)

print(f"\n📦 SRC_ROOT2 병합...")
for s in ["train", "valid", "test"]:
    filter_split(SRC_ROOT2, old_to_new2, s)

print(f"\n📦 SRC_ROOT3 병합...")
for s in ["train", "valid", "test"]:
    filter_split(SRC_ROOT3, old_to_new3, s)
```

STEP 3

의미론적 맥락 학습

식재료를 128 차원 벡터 공간에
투영하는 Ingredient2Vec 도입
재료 간 의미 관계를 수학적으로 파악
최적의 재료 조합 탐색 가능

```
총 alias 매핑 수: 458개

YOLO 클래스 → Canonical 매핑
  ✓ egg                    → egg
  ✓ butter                 → butter
  ✓ onion                  → onion
  ✓ mushroom               → mushroom
  ✓ milk                   → milk
  ✓ garlic                 → garlic
  ✓ tomato                 → tomato
  ✓ bacon                  → Bacon
  ✓ carrot                 → carrot
  ✓ potato                 → potato
  ✓ beef                   → beef
  ✓ mayonnaise             → mayonnaise
  ✓ olive oil              → olive oil
  ✓ cheese                 → cheese
  ✓ chicken                → chicken
  ⚠ Lemon -Nimbu-          → lemon nimbu
  ✓ Capsicum               → Capsicum
  ✓ Brinjal                → Brinjal
  ✓ cabbage                → cabbage
  ✓ cucumber               → cucumber
...
```

출발점 : YOLOv8m 단독 — 그리고 그 한계

선택 이유

- 실시간 객체 탐지에서 가장 널리 쓰이는 계열
- m 사이즈는 속도 · 정확도 균형이 우수
- 다양한 클래스를 빠르게 탐지하는 데 적합

드러난 한계

- 탐지율이 목표 수준에 미달
- 마늘처럼 작은 재료를 자주 놓침
- 채소가 겹쳐있는 상황에서 탐지 실패

단독 모델로는 한계가 명확하다

→ 여러 방향의 보완책을 동시에 실험하기로 결정

YOLO World · YOLO E 앙상블 · YOLOv8m + CLIP

보완책 ① YOLO World · ② YOLOv8n + YOLO E 앙상블

① YOLO World

대규모 라벨링 없이 유연하게 동작 → 기대가 컸음



mAP@50 = 0.0032

텍스트 추론 방식이 파인튜닝 평가와 불일치 · 우유 / 옥수수 탐지 실패

② YOLOv8n + YOLO E 앙상블

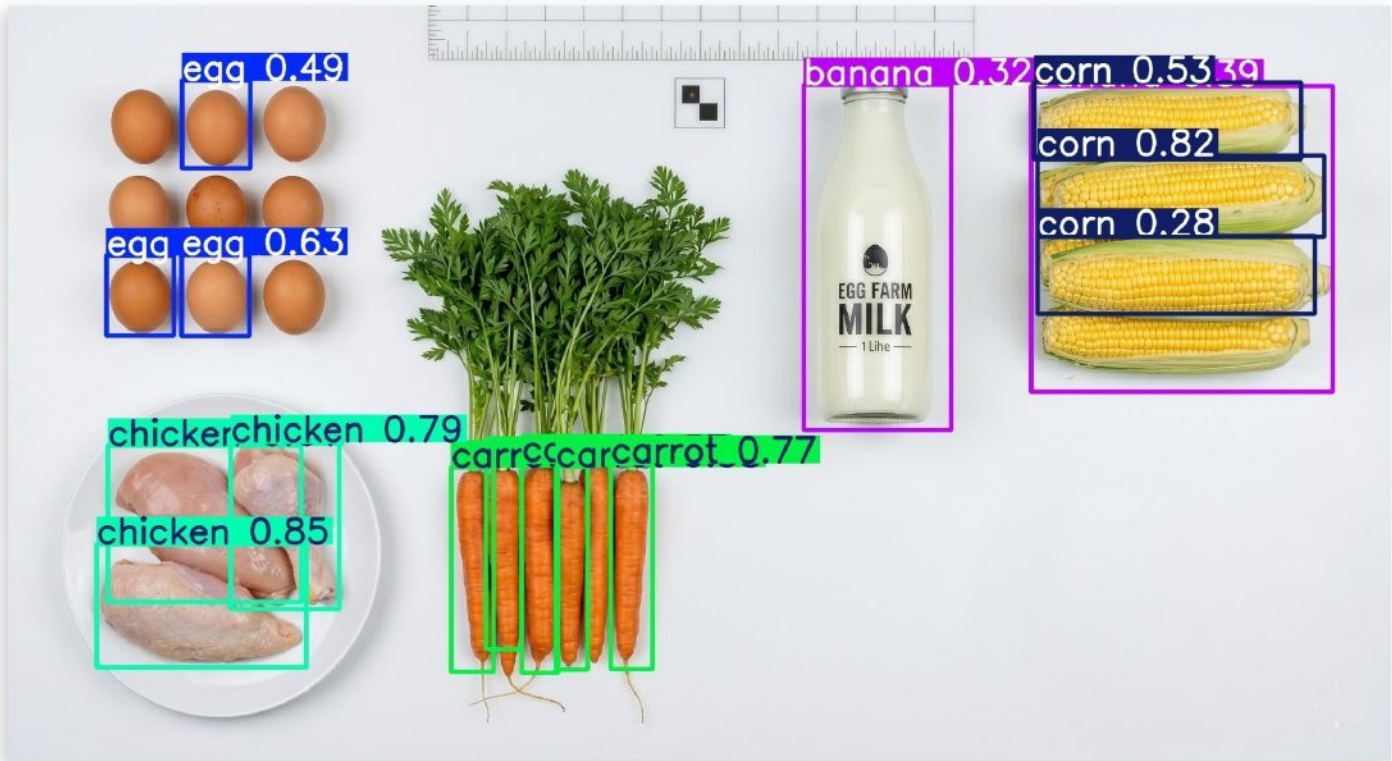
YOLO E 가 놓친 재료를 잡아주길 기대



0.7609 → 0.6779 (오히려 하락)

탐지 기준이 달라 합쳐질 때 오탐 · 노이즈 증가 → 시너지가 아닌 독

보완책 ③ YOLOv8m + CLIP 앙상블



구조

낮은 threshold 로 후보 박스를 늘리고 , 사전학습 CLIP 이 재검증 → 저신뢰도 박스를 살림

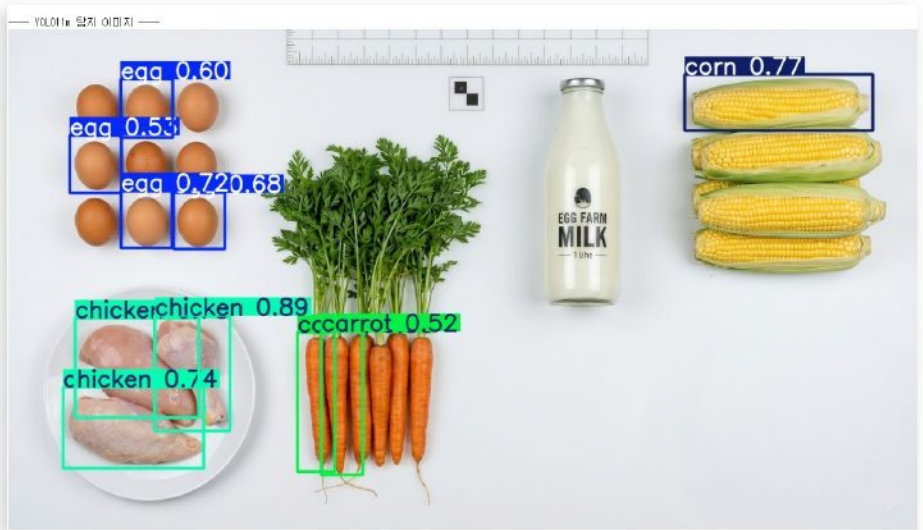
한계

- 순차 추론으로 속도 저하
- YOLO 가 만든 후보 박스 안에서만 작동
- 근본적 탐지율 문제는 미해결

우유를 바나나로 인식 — 오탐은 가장 치명적인 문제

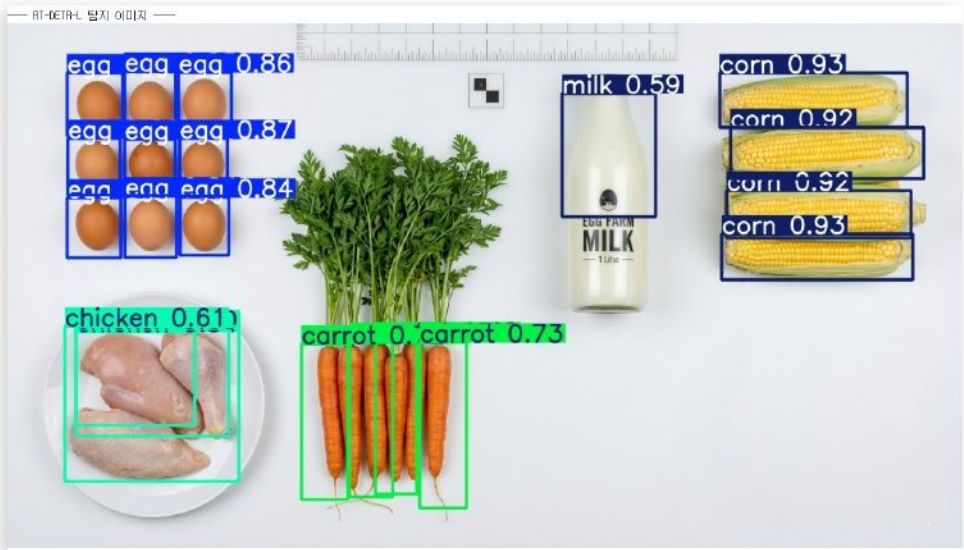
구조적으로 더 강한 단일 모델 : YOLOv11m vs RT-DETR-L

YOLOv11m



50 epoch · mAP@50 0.805 — 이전 시도들보다 크게 향상 .
다만 우유 미탐지 등 한계는 잔존

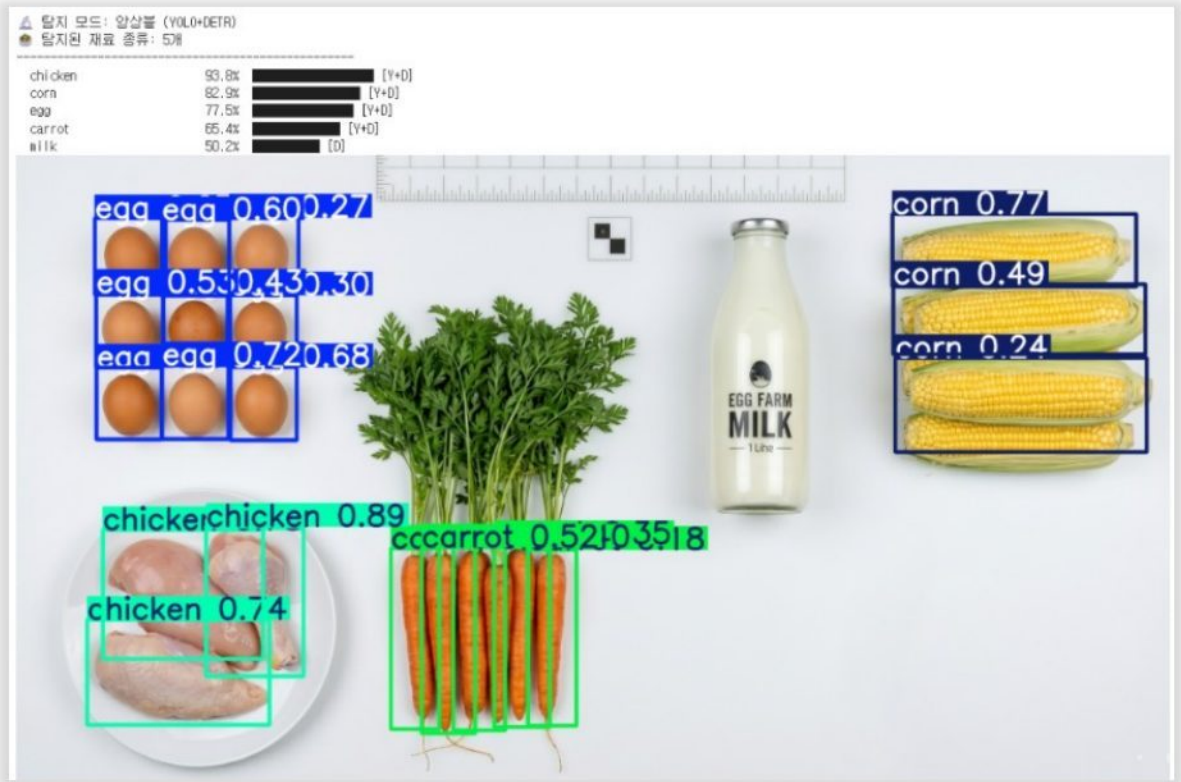
RT-DETR-L



Self-Attention 으로 이미지 전체를 한 번에 스캔 — 모든 식재료를 올바르게 인식

- 전체 맥락 기반 탐지
- **Anchor-free →** 크기 편차에 유연
- **NMS** 후처리 불필요

YOLOv11m + RT-DETR-L 앙상블



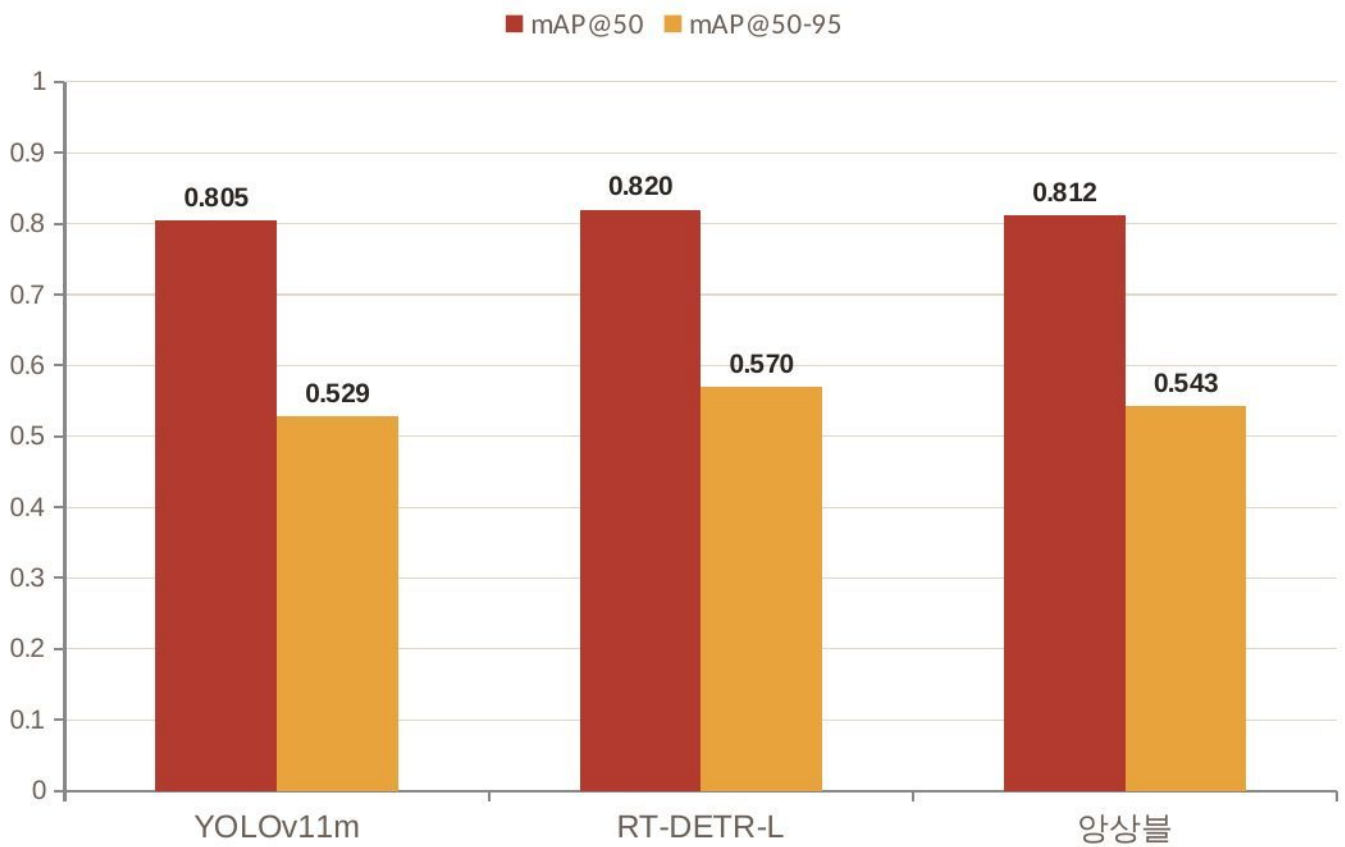
RT-DETR-L 단독으로 사용했을때보다 정확도가 낮아진 모습을 확인할 수 있음 .

| 최종 실험 결과 정리

모델	mAP@50	mAP@50-95
YOLOv11m	0.805	0.529
RT-DETR-L	**0.820**	**0.570**
앙상블	0.812	0.543

RT-DETR-L 이 더 적은 epoch, 더 낮은 해상도임에도 가장 높은 성능을 기록함 .

최종 실험 결과 — RT-DETR-L 단독 선택



RT-DETR-L

더 적은 epoch · 더 낮은 해상도에도 최고 성능

mAP@50-95 +7.8%

다양한 크기 객체 탐지 정밀도에서 유의미한 격차

최종 결론

- 앙상블은 두 모델의 탐지 기준 차이가 노이즈로 작용 → 단독보다 낮음
- RT-DETR-L 이 이미 충분히 높은 성능
- 단순하고 안정적인 단독 모델을 합리적 선택으로 확정

04 RECOMMENDATION

레시피 추천 알고리즘

탐지된 재료가 하나라도 포함된 레시피를 추린 뒤 , 다섯 기준으로 점수화하여 높은 순서로 추천

최종 점수 = 활용률 + 완성도 + 칼로리 근접도 − 부족 재료 패널티 + 채식 보너스

보유 재료 활용률	사용된 내 재료 ÷ 내가 가진 재료 x 40
레시피 완성도	사용된 내 재료 ÷ 레시피 전체 재료 x 40
칼로리 근접도	최적 칼로리 (최소칼로리 + 최대 칼로리)/2) x 10 # 최적 칼로리 중앙값에 가까울수록 높은 점수가 나옴
부족 재료 패널티	부족 재료 ÷ 레시피 전체 재료 x 10
채식 조건 + 채식 레시피	+ 2 점

식단 유형별 칼로리 범위 설계

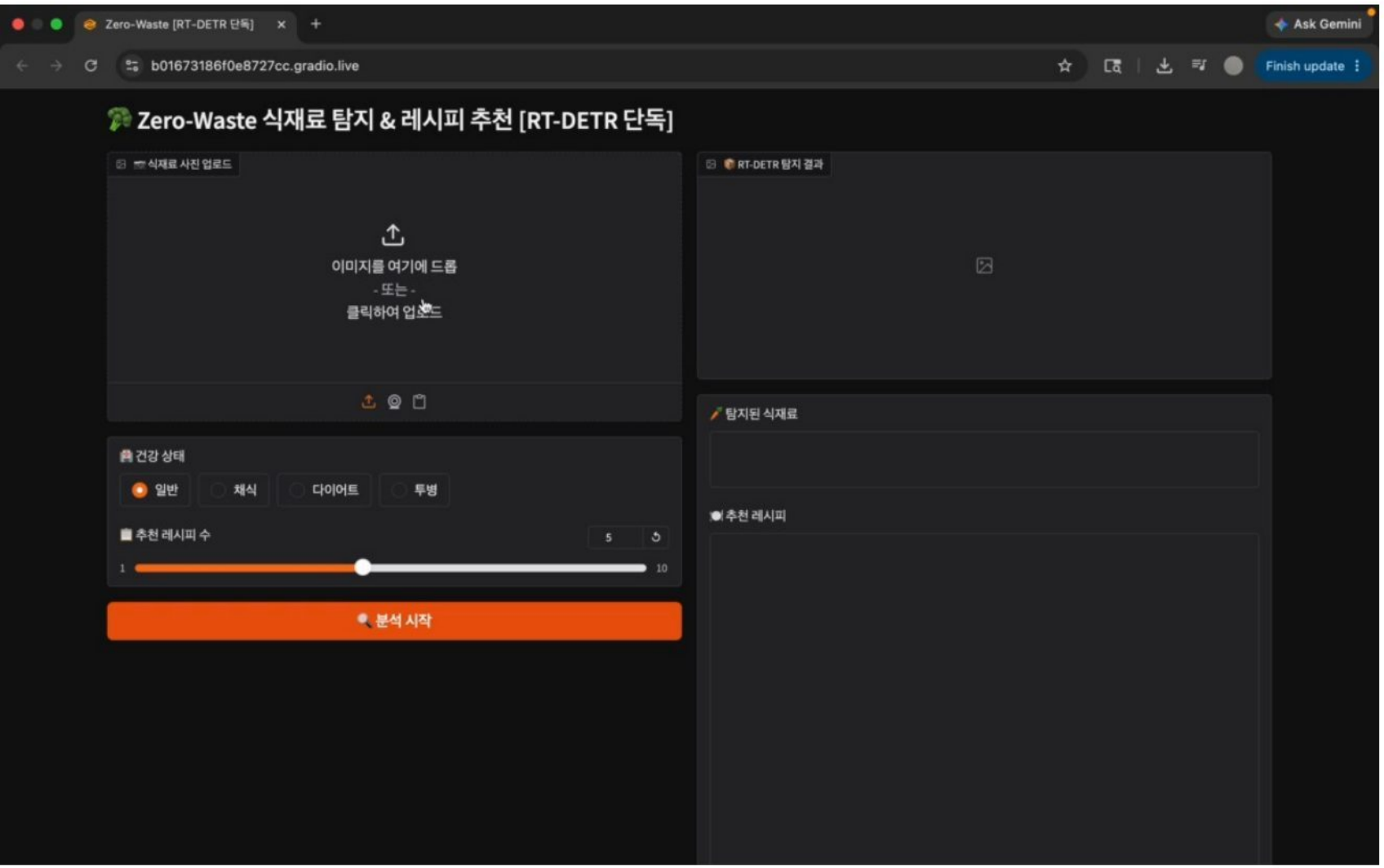
일반	채식	다이어트	투병
350 – 850	**300 – 750**	**200 – 550**	**250 – 700**
kcal	kcal	kcal	kcal
특별한 제한 없는 기본 범위	육류 · 해산물 포함 레시피 제외	저칼로리 레시피가 유리하도록	고칼로리 부담 레시피를 회피
DGA 2020-2025 (1,600~3,000)	*USDA Healthy Vegetarian*	*NEJM 2022 임상연구*	*ESPEN 가이드라인*



채식 모드 제외 재료

chicken · beef · pork · fish · shrimp · salmon · crab · tuna · lamb · turkey · ham · bacon · sausage

Gradio 기반 인터페이스 & 추천 결과



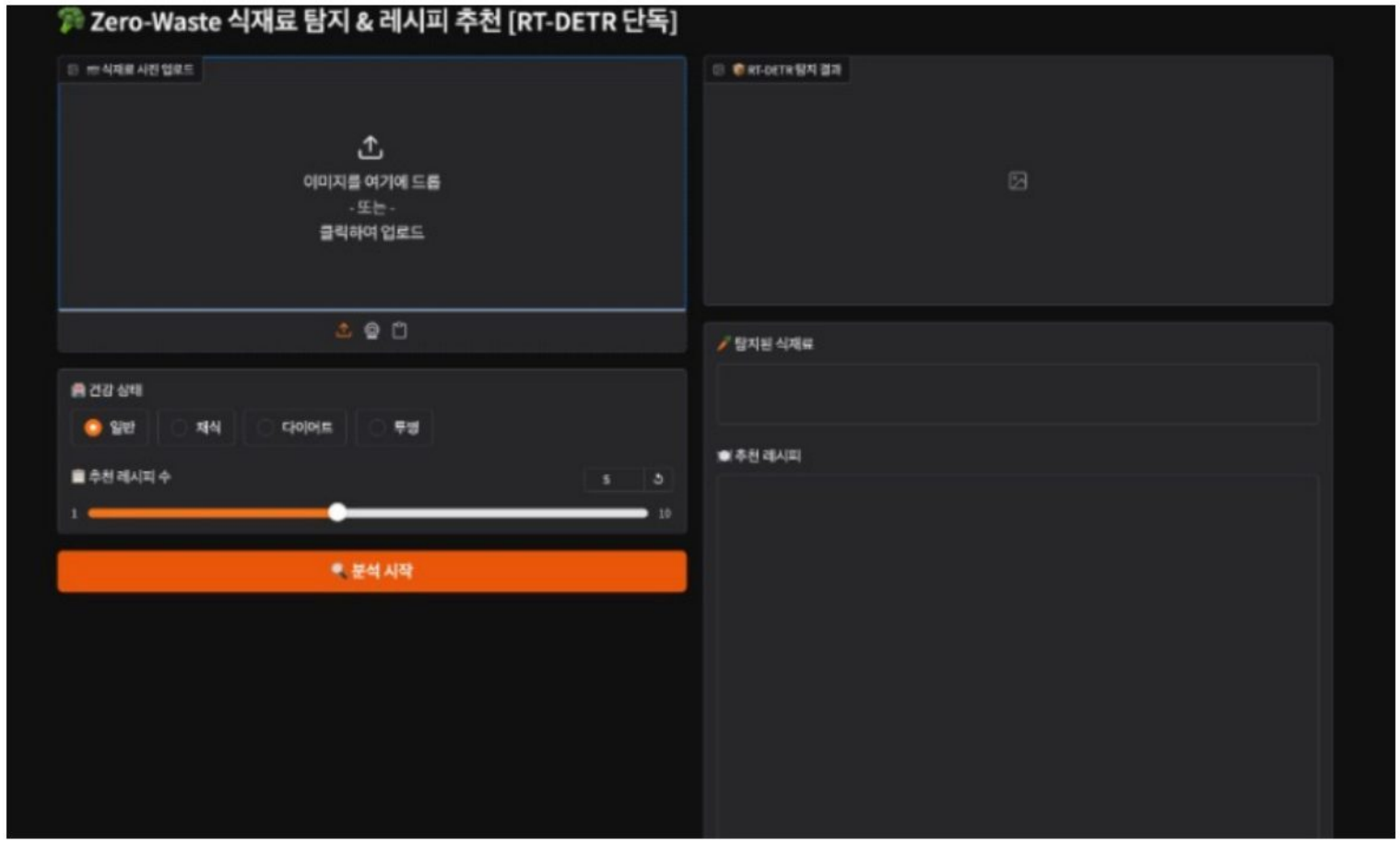
사용 흐름

- 왼쪽 : 사진 업로드 · 식단 유형 · 추천 개수 슬라이더
- 분석 시작 → 오른쪽에 탐지 이미지 출력
- 탐지 재료 목록 + 추천 레시피 함께 표시

실제 업로드 예시

인식된 달걀·시금치·베이컨 등을 활용한 다이어트 레시피가 점수 순으로 추천됨

Gradio 기반 인터페이스 & 추천 결과



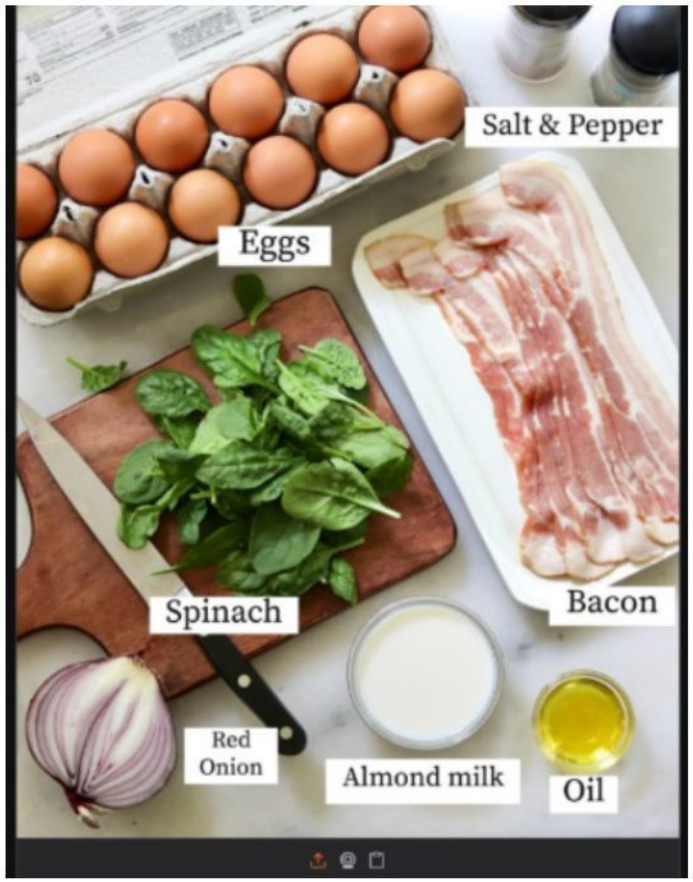
사용 흐름

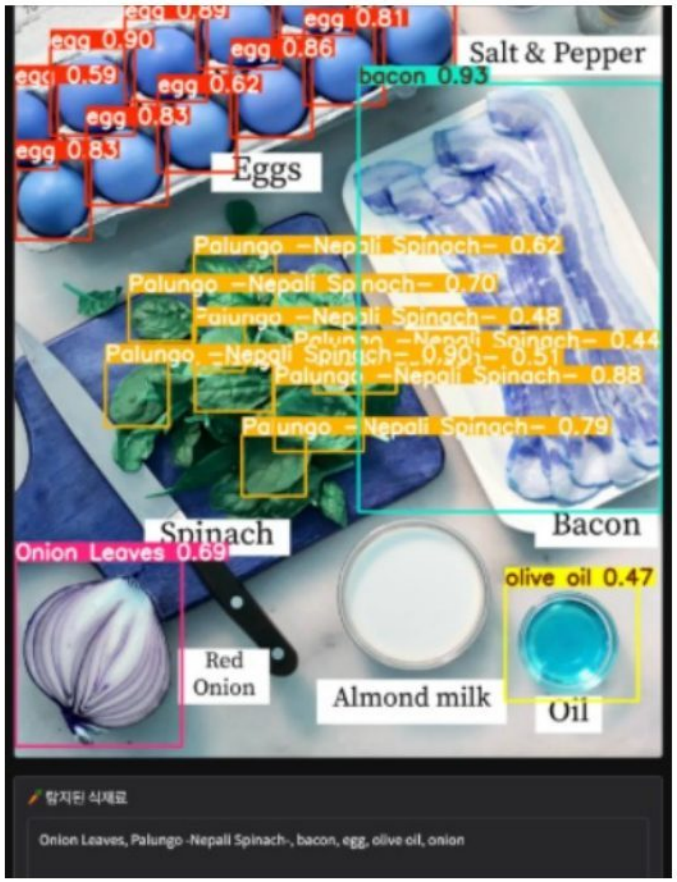
- 왼쪽 : 사진 업로드 · 식단 유형 · 추천 개수 슬라이더
- 분석 시작 → 오른쪽에 탐지 이미지 출력
- 탐지 재료 목록 + 추천 레시피 함께 표시

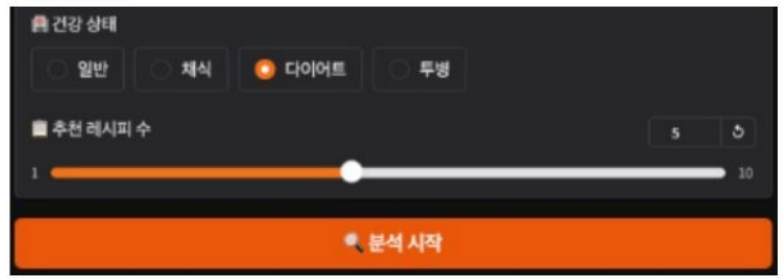
실제 업로드 예시

인식된 달걀·시금치·베이컨 등을 활용한 다이어트 레시피가 점수 순으로 추천됨

Gradio 기반 인터페이스 & 추천 결과



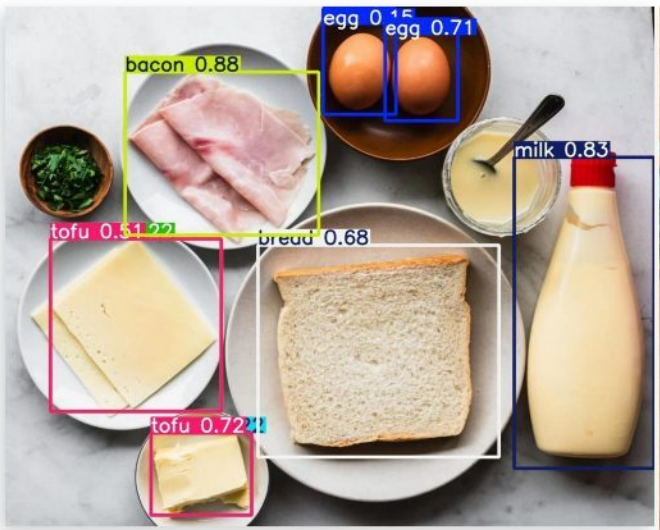


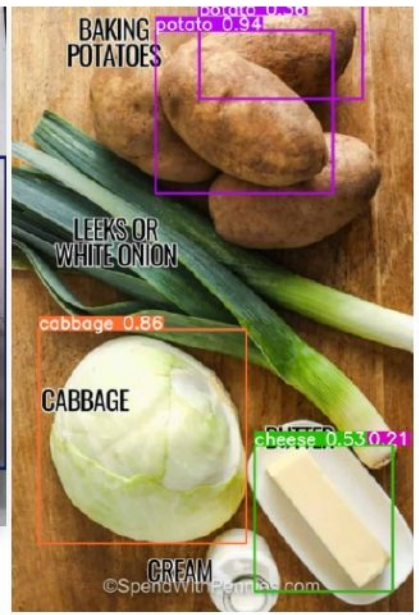




한계 ① 탐지 한계 · ② 데이터 불균형

① RT-DETR 탐지 한계





형태 · 색이 뚜렷한 재료는 잘 탐지하나 ,
버터 · 치즈를 두부로 , 마요네즈를 우유로 혼동함 .

② 데이터 검증의 어려움 · 불균형

구분	클래스명	bbox 수
상위 1	Mushroom	2,076
상위 2	Tomato	1,891
상위 3	Potato	1,750
하위 1	Olive Oil	87
하위 2	Chickpeas	134
하위 3	zucchini	141

학습 예시가 충분한 클래스와 부족한 클래스의 격차가 커 ,
모델 성능 · 추천 결과에 영향을 주었을 가능성

한계 ③ 학습 시간과 자원 — 연산 복잡도



Transformer 의 구조적 비용

- Self-Attention 은 이미지 전체를 한 번에 처리
  → CNN 대비 연산량이 구조적으로 많음
- 복잡도 $O(n^2)$: 해상도↑일수록 제곱으로 증가
- 1280→1536: 토큰 1.44 배 → 연산 약 2 배

YOLO11m	RT-DETR-L
50 ep	35 ep
≈ 6 시간	≈ 7 시간

Colab 컴퓨팅 제한 + 세션 끊김 → Drive 체크포인트로 대응했으나 추가 자원 소모

한계 ④ 추천 알고리즘 — 그리고 극복 방안

추천 알고리즘의 3 가지 한계

1. 가중치의 경험적 설정

반복 테스트로 정한 값일 뿐 , A/B 테스트 · 사용자 실험으로 검증되지 않음

2. 개인화 부재

모든 사용자에게 동일 가중치 → 개인 선호도를 반영하지 못함

3. 사용자 피드백 부재

Food.com 레시피 데이터에만 의존 , 실제 사용 피드백이 없음

극복 방안

협업 필터링 하이브리드

재료 기반 콘텐츠 필터링 + 사용자 행동 기반 협업 필터링
(“ 나와 비슷한 사람”) 을 결합해 추천 품질 향상

사용자 선호도 슬라이더

Gradio UI 에 가중치 조절 슬라이더 추가

→ 활용률 · 칼로리 · 조리시간 중 중요 항목을 직접 설정

CONCLUSION

“냉장고 사진 한 장으로 낭비 없는 식탁을 만드는 AI”

데이터

3 개 데이터셋 통합 + 23 만 레시피 연결 파이프라인 직접 설계

모델

다양한 시행착오 끝에 RT-DETR-L 을 우리 환경 실험으로 검증·선택

서비스

탐지→추천→ Gradio UI 까지 end-to-end 파이프라인 완성

음식물 낭비를 줄이고 개인 건강 상태에 맞는 식단 관리까지 — 피드백을 쌓아 개인화 추천으로 발전 가능

감사합니다 · **Parrot 1 조**